Practical 10 --- Logs You Can Actually Search

Levels, structured JSON events, and what must never be written down

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L18
Course Outcome	CO5
Lab duration	120 minutes
Memory needed	about 350 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Explain why `print()` is not good enough once software is deployed.
2. Use logging levels so you can turn detail up and down without editing code.
3. Write one JSON object per event, so logs can be searched by machine.
4. Recognise what must never appear in a log, and mask it.

What you need to know first

In P08 your service ran inside a container. Suppose at nine at night a customer complains that a prediction was wrong. What do you actually have to look at?

The container is gone. Its filesystem went with it. `print()` output vanished with the process. Y nothing.

That is what **logging** is for. A log is a permanent, timestamped record of what your software did. ideas make one useful.

Levels. Not every message matters equally. Python gives you five: `DEBUG` (detail for develope `INFO` (normal events worth recording), `WARNING` (something odd but survivable), `ERROR` (th request failed), and `CRITICAL` (the service is going down). You set a **threshold** and everythin it is discarded. That is how the same code can be quiet in production and chatty while you are de --- with no code change at all.

Structure. A human reads `"prediction took 42ms"`. A machine cannot. With fifty containe producing thousands of lines an hour, nobody reads logs by eye --- a tool searches them. So wri **JSON** object per line, with named fields: `{"event": "prediction", "duration_ms": 4 "distance_km": 5.0}`. Now "show me every prediction slower than 200 ms" is a query, not afternoon.

Where they go. In a container you do **not** write to a log file. Write to standard output, and let the collect it. That is what `kubectl logs` reads, and what a collector like FluentD or Fluent Bit sh central store such as the ELK stack. A log file inside a container dies with the container, which is the problem you were trying to solve.

One rule that is not technical. A log is a permanent record that many people can read. Names, p numbers, exact home addresses, card numbers --- none of it belongs there. You will practise ren today.

Before you start

- Practical P07 is finished. You have a service with a /predict endpoint.
- You know what a dictionary is and what JSON looks like.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to th one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier c created. If you jump ahead you will see a `NameError`, which just means "you have not ma thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, start again from the first cell. Nothing is damaged by doing this.

In [ ]:
```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn', 'joblib']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn', 'joblib']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
```

```
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the fil
there.

In [ ]:
```
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
```

```
else:
    print("dataset found   ->", DATA)
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Why print() is not enough

Look at what `print` gives you, and what it does not.

In [ ]:
```
print("something went wrong")
```

No time. No severity --- is this fatal or routine? No idea which request, which customer, which co
And no way to turn it off in production without editing the code.

Every one of those is fixed by the logging module.

Step 2 --- Set up a logger

`logging.getLogger(name)` gives you a named logger. We send its output to the screen wi
`StreamHandler`, and decide the format once.

The `handlers.clear()` line matters in notebooks: without it, every time you re-run this cell y
another handler and every message appears twice, then three times.

In [ ]:
```
import logging
import sys
from pathlib import Path

WORK = Path("work")
WORK.mkdir(exist_ok=True)

logger = logging.getLogger("delivery")
logger.setLevel(logging.DEBUG)
logger.handlers.clear()          # re-running this cell is now safe
logger.propagate = False

to_screen = logging.StreamHandler(sys.stdout)
to_screen.setFormatter(logging.Formatter(
    "%(asctime)s  %(levelname)-8s  %(name)s  %(message)s"
))
logger.addHandler(to_screen)

logger.info("logger ready")
```

Step 3 --- The five levels

Run this and read the output. Every line now carries a time and a severity.

In [ ]:
```
logger.debug("feature vector built: [5.0, 20, 2, 0]")
logger.info("prediction served in 3 ms")
logger.warning("distance 47 km is unusually far")
logger.error("model file could not be read")
logger.critical("out of memory, shutting down")
```

Choosing the level is a judgement you will make hundreds of times. The test: **would I want to b up for this?** If yes, ERROR or CRITICAL. If it is just useful history, INFO.

Step 4 --- Turn the volume down without touching the code

This is the payoff of levels. Change one number and the DEBUG and INFO lines disappear --- everywhere, at once.

In [ ]:
```
logger.setLevel(logging.WARNING)

print("--- threshold is now WARNING ---")
logger.debug("you will not see this")
logger.info("nor this")
logger.warning("but you will see this")
logger.error("and this")

logger.setLevel(logging.DEBUG)     # back to chatty for the lab
```

In production you run at INFO. When something is wrong you drop to DEBUG for ten minutes. N rebuild, no redeploy.

Step 5 --- One line, one event, as JSON

Now make the lines machine-readable. A **formatter** decides what a log line looks like; ours build JSON object instead of a sentence.

Anything extra the caller passes lands in the same object, which is what makes the logs searcha

In [ ]:
```
import json
from datetime import datetime, timezone


class JsonFormatter(logging.Formatter):
    """Turn one log record into one line of JSON."""

    def format(self, record):
        event = {
            "time": datetime.fromtimestamp(
                record.created, timezone.utc
            ).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
        }
        # anything passed as extra={...} is added here
        for key, value in getattr(record, "fields", {}).items():
            event[key] = value
        return json.dumps(event)
```

```
def log_event(level, message, **fields):
    """Log one message with named fields attached."""
    logger.log(level, message, extra={"fields": fields})


to_screen.setFormatter(JsonFormatter())

log_event(logging.INFO, "prediction served",
          duration_ms=3, distance_km=5.0, predicted_min=42.1)
```

Read that line. Every fact has a name. A search tool can now answer "average duration_ms yest without anyone writing a parser.

Step 6 --- Send the same events to a file as well

On your laptop it is convenient to keep a copy on disk so you can search it in Step 8. **In a contai would not do this** --- you would log only to the screen and let the platform collect it.

The file format is **JSONL**: one JSON object per line.

In [ ]:
```
LOG_FILE = WORK / "service.log"
LOG_FILE.unlink(missing_ok=True)

to_file = logging.FileHandler(LOG_FILE, encoding="utf-8")
to_file.setFormatter(JsonFormatter())
logger.addHandler(to_file)

log_event(logging.INFO, "service started", version="1.0.0")
print("logging to", LOG_FILE)
```

Step 7 --- Log a service doing its job

Now simulate 40 predictions and log each one properly: what came in, what went out, how long Two of them go wrong on purpose.

Notice `request_id`. When something is reported hours later, that id is how you find the one li among thousands that belongs to it.

In [ ]:
```
import random
import time
import uuid

import joblib
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
orders = pd.read_csv(DATA)
X_train, X_test, y_train, y_test = train_test_split(
    orders[FEATURES], orders["delivery_min"],
    test_size=0.2, random_state=42,
)
```

```
model = LinearRegression().fit(X_train, y_train)

rng = random.Random(42)
to_screen.setLevel(logging.WARNING)   # keep the notebook readable

for i in range(40):
    request_id = str(uuid.uuid4())[:8]
    row = X_test.iloc[[i]]
    started = time.perf_counter()

    if i in (11, 27):                 # two deliberate failures
        log_event(logging.ERROR, "prediction failed",
                  request_id=request_id,
                  error="ValueError: traffic_level out of range",
                  duration_ms=round(
                      (time.perf_counter() - started) * 1000, 2))
        continue

    minutes = float(model.predict(row)[0])
    duration = (time.perf_counter() - started) * 1000
    if rng.random() < 0.15:           # a few slow ones
        duration += rng.uniform(200, 400)

    log_event(logging.INFO, "prediction served",
              request_id=request_id,
              distance_km=float(row["distance_km"].iloc[0]),
              traffic_level=int(row["traffic_level"].iloc[0]),
              predicted_min=round(minutes, 1),
              duration_ms=round(duration, 2))

to_screen.setLevel(logging.NOTSET)
print("40 requests logged to", LOG_FILE)
print("lines written:",
      len(LOG_FILE.read_text(encoding='utf-8').splitlines()))
```

Only the two errors appeared on screen, because we raised the screen handler's threshold to WARNING. Everything still went to the file. Two destinations, two levels, one line of code each.

Step 8 --- Search the logs like a machine would

This is the whole reason for JSON. Read the file back and answer real questions with ordinary P

In [ ]:
```
events = [json.loads(line)
          for line in LOG_FILE.read_text(encoding="utf-8").splitlines()
          if line.strip()]

print("total events :", len(events))

errors = [e for e in events if e["level"] == "ERROR"]
print("errors       :", len(errors))

served = [e for e in events if e.get("message") == "prediction served"]
durations = [e["duration_ms"] for e in served]
print("predictions  :", len(served))
print("average ms   :", round(sum(durations) / len(durations), 1))

slow = [e for e in served if e["duration_ms"] > 200]
print("slower than 200 ms:", len(slow))
```

```
print()
print("The slowest request:")
worst = max(served, key=lambda e: e["duration_ms"])
print(f"  request_id {worst['request_id']}  "
      f"{worst['duration_ms']} ms  "
      f"{worst['distance_km']} km")
```

Every one of those questions would have been an afternoon of squinting at sentences. That is th difference structure makes.

Step 9 --- What must never go in a log

Logs are kept for months and read by many people. Some things must never be written into one

- names, phone numbers, exact home addresses
- card numbers, passwords, API keys, tokens
- anything identifying one individual customer

You almost never need the value. You need to be able to *match* it. A one-way hash lets you say ' two orders came from the same customer" without recording who that is.

In [ ]:
```
import hashlib

bad_order = {
    "customer_name": "Ananya Sharma",
    "phone": "9876543210",
    "address": "Flat 402, Green Park, Noida",
    "distance_km": 5.0,
    "traffic_level": 2,
}


def safe_for_logging(order):
    """Keep what helps debugging, remove what identifies a person."""
    secret = str(order.get("phone", ""))
    return {
        "customer_ref": hashlib.sha256(
            secret.encode()).hexdigest()[:12],
        "distance_km": order["distance_km"],
        "traffic_level": order["traffic_level"],
    }


print("NEVER log this:")
print("  ", bad_order)
print()
print("Log this instead:")
print("  ", safe_for_logging(bad_order))

log_event(logging.INFO, "order received",
          **safe_for_logging(bad_order))
```

`customer_ref` is stable: the same phone always gives the same reference, so you can still g customer's orders. But nobody reading the log learns a phone number.

Step 10 --- Where logs go once you are in a cluster

Finally, the operational picture from L18. You do not have to run any of this today --- you have to to draw it.

In [ ]:
```
pipeline = [
    ("your code", "writes one JSON line per event to stdout"),
    ("the container runtime", "captures stdout to a file on the node"),
    ("a collector (Fluent Bit / FluentD)",
     "runs on every node and reads those files"),
    ("a store (Elasticsearch / OpenSearch)",
     "indexes the events so they can be searched"),
    ("a viewer (Kibana / Grafana)", "is where a human asks questions"),
]

for i, (part, does) in enumerate(pipeline, 1):
    print(f"  {i}. {part}")
    print(f"     {does}")

print()
print("Two rules that follow from this picture:")
print("  - never write logs to a file inside a container;")
print("    the container dies and takes the file with it")
print("  - never log secrets; that store is searchable by")
print("    everyone in the company")
```

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you whic are correct. You can run it as many times as you like.

Task T1 --- Decide what gets recorded

```
A colleague wants the service to record warnings and errors only --
nothing routine.

Set the `logger` threshold so that `DEBUG` and `INFO` are discarded
but
`WARNING`, `ERROR` and `CRITICAL` are kept. Then build a list
`T1_kept` naming the levels that would still be recorded, as upperc
strings.
```

> **Hint.** `logger.setLevel(logging.WARNING)` sets the threshold. For the list, use `logger.isEnabledFor(logging.DEBUG)` and so on to ask the logger itself, rather than typing the answer from memory.

In [ ]:
```
LEVELS = {"DEBUG": logging.DEBUG, "INFO": logging.INFO,
          "WARNING": logging.WARNING, "ERROR": logging.ERROR,
          "CRITICAL": logging.CRITICAL}

# TODO: set the logger threshold so only WARNING and above survive

# TODO: which level names would still be recorded?
T1_kept = None

print("still recorded:", T1_kept)
```

Task T2 --- A formatter for the delivery service

Write a class `ServiceFormatter` (a `logging.Formatter`) whose `format`
method returns one line of JSON with **exactly** these five keys:

- `"ts"` --- the time, as an ISO string
- `"level"` --- the level name
- `"service"` --- always the string `"delivery-api"`
- `"msg"` --- the message
- `"env"` --- always the string `"lab"`

Then create one instance of it, call `format` on a test record, and store the **parsed dictionary** in `T2_line`.

> **Hint.** Copy `JsonFormatter` from Step 5 and change the keys. To make a record to te with, the stub gives you `sample_record`. Call `ServiceFormatter().format(sample_record)` to get a string, then `json.loads(...)` it.

In [ ]:
```
sample_record = logger.makeRecord(
    "delivery", logging.INFO, "x", 1, "prediction served", None, None
)


class ServiceFormatter(logging.Formatter):
    def format(self, record):
        # TODO: return one line of JSON with the five required keys
        return "{}"


# TODO: format the sample record and parse it back into a dictionary
T2_line = None

print(T2_line)
```

Task T3 --- Answer a question from the logs

```
Your manager asks: **"of the predictions we served, what fraction t
longer than 100 milliseconds, and what was the longest one?"**

Using the `events` list from Step 8, work it out:

- `T3_slow_fraction` --- the fraction (between 0 and 1) of
  `"prediction served"` events with `duration_ms` above 100
- `T3_worst_id` --- the `request_id` of the slowest one
```

> **Hint.** Filter `events` down to those whose `message` is `"prediction served"`. Th fraction is `len(slow) / len(served)`. For the slowest, use `max(served, key=...)` on `duration_ms` and read its `request_id`.

In [ ]:
```
served_events = [e for e in events
                 if e.get("message") == "prediction served"]

# TODO: what fraction took longer than 100 ms?
T3_slow_fraction = None

# TODO: the request_id of the slowest one
T3_worst_id = None

print("fraction over 100 ms:", T3_slow_fraction)
print("slowest request     :", T3_worst_id)
```

Self-check

Run the cell below to mark your work.

In [ ]:
```
# ----------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ----------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | the logger threshold is WARNING', lambda: logger.level == lo
_check('T1 | T1_kept names exactly the three surviving levels', lambda: s
_check('T2 | the line has exactly the five required keys', lambda: set(T2
```

```
_check('T2 | service and env carry the fixed values', lambda: T2_line['se
_check('T2 | level and message come from the record', lambda: T2_line['le
_check('T2 | it still works for a different record', lambda: (lambda d: d
_check('T3 | T3_slow_fraction matches the log file', lambda: abs(float(T3
_check('T3 | T3_worst_id is the slowest served request', lambda: T3_worst

print("======================================================================
print("SELF-CHECK   Practical 10 --- Logs You Can Actually Search")
print("======================================================================
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------------
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("======================================================================
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell agai
```

What to submit

1. This notebook, with every cell run and its output visible.
2. Your `work/service.log`, showing at least 40 JSON lines.
3. In a markdown cell, three fields you would add to every log line in a real delivery service, an each one helps.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, log file produced	3
Task T1 --- levels used to control what is recorded	2
Task T2 --- a formatter with the required fields	3
Task T3 --- a real question answered from the logs	2
Total	**10**

Read more

- Python --- Logging HOWTO --- https://docs.python.org/3/howto/logging.html
- Python --- Logging cookbook --- https://docs.python.org/3/howto/logging-cookbook.html
- Kubernetes --- Logging architecture --- https://kubernetes.io/docs/concepts/cluster-administ logging/
- The twelve-factor app --- Logs --- https://12factor.net/logs